Documentación del Proyecto Compilador

Etapa 5: Generación de Código Assembly (x86-64) y Testing

1 División del Trabajo

El desarrollo de la quinta etapa del compilador, centrada en el diseño e implementación del generador de Código Ensamblador x86-64 (`assembly`), el soporte para funciones externas (`extern`), su integración con el gestor de compilación y la elaboración de pruebas de ejecución, se realizó de manera colaborativa distribuyendo las responsabilidades de la siguiente forma:

- **Diseño e implementación del Generador de Código Assembly (`assembly.c/.h`):** Gribaudo Re Francisco y Francisco Andreani.
- **Manejo del compilador, CLI y soporte de métodos externos (`compiler-manager.c/.h`, `cli.c`, `parser.y`, `codinter.c`):** Valentín Pastre Thuer.
- **Elaboración de pruebas unitarias y de integración (`test_assembly.c`):** Francisco Andreani y Gribaudo Re Francisco.

2 Decisiones de Diseño, Suposiciones y Aclaraciones

Habiendo concluido en la etapa anterior del compilador mediante la generación de Código Intermedio de Tres Direcciones, el objetivo de esta etapa consistió en utilizar el codigo de tres dirrecciones para construir el código ensamblador **x86-64 (GCC en Linux)**.

- **Adopción de la Convención de Llamadas:** Para permitir enlazar funciones escritas en C (como los ejemplos: `get_int` o `printInt`), se respetó el estándar de la arquitectura x86-64. Los primeros seis argumentos de una invocación se pasan a través de los registros `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` y `%r9`, mientras que el valor de retorno se deposita en el registro `%rax`.
- **Asignación en el Marco de Pila (*Stack Frame*) y Registros de Trabajo:** Se optó por un esquema de almacenamiento uniforme donde todos los parámetros formales, variables locales y variables temporales (`t0, t1, t2...`) de una función reciben una casilla dedicada de 8 bytes (*quadword*) con desplazamiento negativo respecto al puntero base `%rbp` (`-8(%rbp)`, `-16(%rbp)`, etc.). Para ejecutar las operaciones sin corromper los registros de argumentos ni el puntero de pila, todas las instrucciones cargan sus operandos en los registros temporales `%r10` y `%r11` (o `%rax` y `%rdx` en el caso de la división y el módulo), operan entre registros y vuelcan el resultado nuevamente a memoria.
- **Soporte de Declaraciones `extern`:** Se incorporó el reconocimiento de métodos declarados con el calificador `extern` en el análisis léxico-sintáctico (representados en el AST como nodos `function` con hijo derecho `NULL`). De este modo, el analizador

semántico valida sus firmas en la Tabla de Símbolos, pero tanto `codinter.c` como `assembly.c` omiten la generación de cuerpo para dichas funciones, emitiendo únicamente la instrucción `call <nombre>` cuando son invocadas.

3 Diseño y Decisiones Clave: El Generador de Código Assembly

El desarollo de esta entrega se vio principalmente en `src/assembly.c` y `header/assembly.h`, donde se implementaron las estructuras de administración de memoria y el traductor de instrucciones de tres direcciones:

- **Gestión de Variables Globales (`registerGlobal`, `isGlobalVar`, `collectGlobalsAndEmit`, `clearGlobalVars`):**
  - Antes de traducir las funciones, `collectGlobalsAndEmit` recorre recursivamente el subárbol de declaraciones globales (`astRoot->left`). Por cada nodo `varDeclaration`, registra el identificador en la lista enlazada `globalHead` mediante `registerGlobal` y emite la directiva `.comm <nombre>, 8, 8`, reservando 8 bytes alineados a 8 bytes en la sección BSS.
  - La función `isGlobalVar` permite distinguir en tiempo de traducción si un identificador debe accederse mediante direccionamiento relativo al contador de programa (`<id>(%rip)`) en lugar de buscarlo en la pila local.
- **Construcción y Alineación del Marco de Pila (`getOrAssignOffset`, `buildFunctionStackFram` `clearStackVars`):**
  - La función `getOrAssignOffset(Symbol *s)` busca el nombre del símbolo en la lista local `stackHead`; si ya fue registrado, devuelve su desplazamiento negativo actual. Si no existe (y no es un literal, etiqueta, función ni variable global), decrementa `currentStackOffset` en 8 bytes, inserta un nuevo nodo `StackVar` y retorna el nuevo offset.
  - Al encontrar el inicio de una función (`Codinter_FUNC_BEGIN`), `buildFunctionStackFrame` limpia la tabla local anterior, asigna offsets a los parámetros formales de la función (`funcSym->paramsNode`) y escanea todas las cuádruplas hasta `Codinter_FUNC_END` registrando cada variable local y temporal utilizada en `arg1`, `arg2` y `result`.
  - Finalmente, calcula el tamaño total del marco (`frameSize = -currentStackOffset`) y verifica `if (frameSize % 16 != 0) frameSize += 8;`, garantizando que el puntero de pila `%rsp` quede estrictamente alineado a múltiplos de 16 bytes como exige la ABI de x86-64 antes de cualquier instrucción `call`.
- **Funciones Auxiliares de Carga y Almacenamiento (`loadOperand`, `storeOperand`):**
  - `loadOperand(Symbol *s, const char *reg, FILE *out)` abstrae el origen del dato emitiendo `movq $<val>, <reg>` para literales enteros o booleanos (`1` o `0`), `movq <id>(%rip), <reg>` para variables globales y `movq <offset>(%rbp), <reg>` para variables locales, parámetros o temporales.
  - `storeOperand(const char *reg, Symbol *dest, FILE *out)` vuelca el valor calculado en el registro hacia `<id>(%rip)` si el destino es global, o hacia `<offset>(%rbp)` si pertenece al marco de pila.

- **Traducción de Instrucciones de Tres Direcciones (`generateAssembly`):**
  - **Prólogo y Epílogo de Funciones (`Codinter_FUNC_BEGIN`, `Codinter_FUNC_END`, `Codinter_RETURN`):** Al iniciar cada función se emite la directiva `.globl <id>`, la etiqueta de entrada, la creación del marco (`pushq %rbp`, `movq %rsp, %rbp`, `subq $<frameSize>, %rsp`) y se copian los parámetros recibidos en `argRegs` (`%rdi` a `%r9`) hacia sus respectivas posiciones en la pila. En `Codinter_RETURN`, se carga el valor de retorno en `%rax` (o `$0` si es vacío) seguido de `leave` y `ret`.
  - **Operaciones Aritméticas (`ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `NEG`):** Para suma, resta y multiplicación se cargan los operandos en `%r10` y `%r11`, aplicando `addq`, `subq` o `imulq` sobre `%r10`. Para `DIV` y `MOD`, se carga el dividendo en `%rax`, se extiende su signo a 128 bits en `%rdx:%rax` con la instrucción `cqto`, se divide por `%r11` con `idivq %r11` y se almacena `%rax` (cociente) o `%rdx` (resto) según corresponda. La resta unaria (`NEG`) emplea `negq %r10`.
  - **Operaciones Lógicas y Relacionales (`AND`, `OR`, `NOT`, `LT`, `GT`, `EQ`):** Los operadores booleanos binarios utilizan `andq` y `orq`, mientras que la negación lógica (`NOT`) invierte el bit menos significativo mediante `xorq $1, %r10`. Las comparaciones relacionales ejecutan `cmpq %r11, %r10`, capturan el estado de los *flags* en el byte inferior `%al` con `setl`, `setg` o `sete`, y lo expanden con ceros a 64 bits mediante `movzbq %al, %r10`.
  - **Control de Flujo e Invocaciones (`LABEL`, `GOTO`, `IF_FALSE`, `PARAM`, `CALL`):** Las etiquetas de salto se emiten con prefijo de etiqueta local (`.<L>:`) y los saltos incondicionales con `jmp .<L>`. La instrucción `IF_FALSE` compara la condición con cero (`cmpq $0, %r10`) y salta si es igual mediante `je .<L>`. Los nodos `PARAM` encolan los símbolos en el arreglo `pendingParams`, los cuales son cargados en orden sobre los registros `argRegs` justo antes de emitir `call <func>` en `Codinter_CALL`, almacenando finalmente `%rax` en el temporal de destino si la función retorna un valor. Al finalizar el archivo se añade la directiva `.section .note.GNU-stack,"",@progbits` para indicar al enlazador que la pila no requiere permisos de ejecución.

4 Integración con el Resto del Compilador

Para integrar la generación de código ensamblador como etapa final del compilador se actualizaron los siguientes módulos:

- **Gestor de Compilación (`compiler-manager.c/.h`):** Se incluyó la cabecera `assembly.h` y se completó la función `assemblyStage()`. Esta función ejecuta en secuencia el análisis sintáctico (`yyparse()`), el análisis semántico (`analyzeAst(syntaxTree, st)`), la generación de código intermedio (`generateCodinterFromAST(syntaxTree)`) y finalmente invoca a `generateAssembly(syntaxTree, getCodinterListHead(), outputFilePtr)` volcando el código x86-64 en el archivo con extensión `.ass` creado por `makeOutputFile()`.
- **Interfaz de Línea de Comandos (`cli.c/.h`):** En `returnTargetStage()` se actualizó el comportamiento por defecto para retornar la etapa `assembly` cuando no se especifica la bandera `-target`, además de reconocer explícitamente `-target assembly`.

5 Instrucciones de Compilación y Ejecución de Pruebas

El proyecto mantiene la integración con **Ceedling** (Unity + CMock) para la validación automatizada de todas las etapas y la compilación del binario `c-tds`.

5.1. Ejecución de Tests Unitarios y de Hardware con Ceedling

En esta etapa se implementó la suite `test/test_assembly.c`, la cual compila programas completos en C-TDS hasta generar el archivo `.ass`, invoca a `gcc -x assembler` para verificar que el código ensamblador sea 100% válido, ejecuta el binario nativo resultante y comprueba el valor devuelto en `%rax` mediante `WEXITSTATUS`:

- **Correr tests del Generador de Código Assembly:**

  ```
  ./ceedling test:assembly
  ```

- **Correr la batería completa de tests del proyecto:**

  ```
  ./ceedling test:all
  ```

5.2. Construcción y Uso del Compilador (Generación y enlace de archivos `.ass`)

Para compilar la versión de lanzamiento (*Release*), generar el código ensamblador de un archivo fuente `.ctds` y ensamblarlo con GCC:

```
./compile.sh
./build/artifacts/release/c-tds -target assembly -o salida prueba.ctds
gcc -x assembler salida.ass -o programa
./programa
```

En caso de utilizar funciones externas declaradas con `extern` (como `get_int` y `printInt` implementadas en un archivo auxiliar `libreria.c`), el enlace se realiza ejecutando:

```
gcc -x assembler salida.ass -x c libreria.c -o programa_final
./programa_final
```

6 Problemas Conocidos y Testing

Durante el desarrollo y testeo de esta quinta etapa se identificaron y resolvieron los siguientes aspectos técnicos:

- **Orden de Evaluación en Llamadas a Funciones Anidadas:** Al probar expresiones con llamadas anidadas del estilo `y = inc(get_int());`, se observó que si se emitían las instrucciones `Codinter_PARAM` a medida que se recorrían los argumentos, los parámetros de una función externa podían mezclarse con los de una llamada interna. Para solucionarlo, en `generateExprCodinter` (`case functionInvoke`) primero se evalúan recursivamente todas las expresiones de los argumentos actuales guardando sus símbolos temporales en un arreglo local (`argSyms`), y luego se emiten todas las instrucciones `Codinter_PARAM` juntas inmediatamente antes de `Codinter_CALL`.

- **Aislamiento de Marcos de Pila en Llamadas Recursivas:** Para verificar que la reserva dinámica del stack (`subq $<frameSize>, %rsp`) y el guardado de parámetros desde `%rdi` hacia `-8(%rbp)` no sobrescribieran valores entre invocaciones sucesivas, se incorporó un caso de prueba de factorial recursivo en `test_assembly.c`, comprobando la correcta preservación del contexto en cada nivel de recursión.